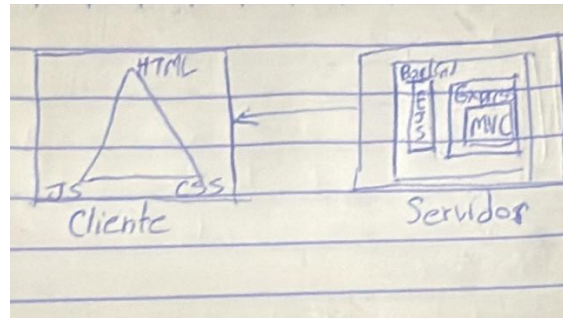


Evidencia – STC0204 Desarrollo de Componentes de Software

Durante toda la unidad de formación (y hasta el momento) he logrado conseguir aprender y entender de manera concisa el funcionamiento y la estructura de una aplicación web, desde el lado del cliente al lado del servidor (monolito de operaciones)



Además de entender el funcionamiento y la estructura también aprendí la manera de hacer que la estructura de un proyecto se haga realidad, desde estilizar una página para que sea visualmente cómoda hasta desplegar un sistema, donde no fue un camino fácil, debido a las múltiples formas en que desarrollan estos sistemas que cada módulo cuenta con especializaciones.

Componentes de un Sistema de Software:

Front-end:

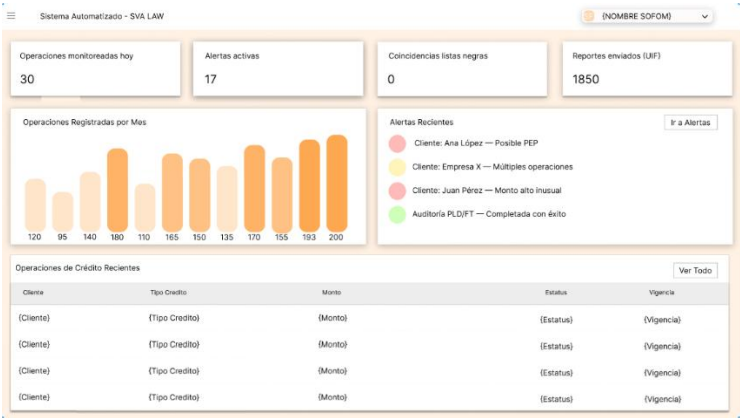
Este componente es la forma en que un usuario va a ver el sistema, además de que será donde este va a interactuar.

Contribuciones a Front-end en mi proyecto:

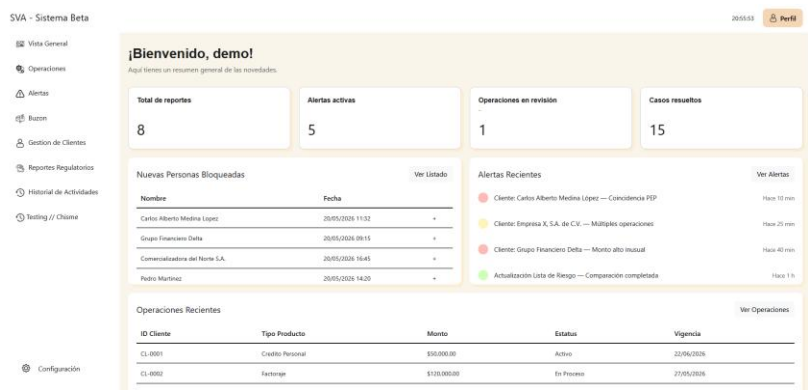
Durante el paso de los avances la vista de algunas visualizaciones han sido cambiadas, esto debido a cambios en la necesidad o en el enfoque y la necesidad que van a cumplir al socio, por ejemplo: durante el desarrollo de WireFrames en las primeras sesiones tuvimos una idea de lo que íbamos a mostrar en el dashboard, posteriormente esta idea fue cambiando, elementos como el color principal y la información que proporciona al socio.

Aquí se presenta un ejemplo de cual fue la idea principal del equipo durante las primeras sesiones hasta el despliegue:

Dashboard desarrollado en Figma (Wireframe):



Dashboard en despliegue:



Estos cambios surgieron de diferentes necesidades y retroalimentaciones que nuestros socios formadores nos propusieron, además de que el diseño anterior era ineficiente, debido a que para llegar a cualquier destino sumaba 1click (Click de abrir sidebar), que puede resultar que llegar a los distintos destinos sea tardado y difícil.

Backend:

Este componente es donde los distintos módulos que dan funcionamiento al sistema están.

En el back-end de mi proyecto participé en el desarrollo de distintos componentes relacionados con la lógica del sistema, la conexión con la base de datos, la creación de rutas, las consultas SQL, el funcionamiento de botones y la integración entre las vistas del sistema.

Mi participación consistió en apoyar en la estructura del servidor, configurando rutas que permitieran comunicar el front-end con el back-end. Estas rutas fueron esenciales para que las acciones realizadas por el usuario, como presionar botones, enviar formularios o acceder a diferentes secciones del sistema, pudieran ejecutarse correctamente.

También trabajé en la conexión con la base de datos, permitiendo que el sistema pudiera consultar o registrar información según las necesidades del proyecto. Para esto se utilizaron consultas SQL y funciones del servidor que procesan los datos recibidos desde la interfaz antes de enviarlos a la base de datos.

Además, contribuí en el funcionamiento de botones y referencias dentro del sistema, asegurando que cada acción estuviera conectada con la ruta correspondiente. Esto permitió que los usuarios pudieran navegar entre secciones, ejecutar procesos y visualizar información de manera correcta.

Estas fueron mis contribuciones dentro del back-end. Un ejemplo de ello es la función `addCliente`, la cual se ejecuta cuando el oficial da click en el botón para agregar un nuevo cliente. Esta acción despliega un panel con un formulario donde se capturan los datos del cliente. Al guardar la información, el sistema envía una solicitud al back-end, donde se procesan los datos y se realiza el registro en la base de datos. La base de datos asigna automáticamente el id del cliente y el estatus inicial como activo. Finalmente, el cliente registrado se agrega a la vista del oficial, permitiendo que la información se pueda visualizar inmediatamente dentro del sistema.

```
20 export.addCliente = async (req, res) => {
21   const {
22     nombre,
23     tipo_persona,
24     rfc,
25     domicilio,
26     correo,
27     telefono,
28     estatus,
29     motivo_bloqueo,
30     fecha_bloqueo
31   } = req.body;
32
33   const camposObligatorios = [
34     nombre,
35     tipo_persona,
36     rfc,
37     domicilio,
38     correo,
39     telefono,
40     estatus
41   ];
42   const tiposPermitidos = ["Física", "Moral"];
43   const estatusPermitidos = ["Activo", "Bloqueado"];
44   const limites = {
45     nombre: 100,
46     rfc: 13,
47     domicilio: 150,
48     correo: 100,
49     telefono: 20,
50     motivo_bloqueo: 200
51   };
52   const excedeLimite = Object.entries(limites).some(([campo, limite]) => {
53     const valor = req.body[campo];
54     return typeof valor === "string" && valor.trim().length > limite;
55   });
56
57   if (
58     camposObligatorios.some((campo) => !req.body[campo].trim()) ||
59     !tiposPermitidos.includes(tipo_persona) ||
60     !estatusPermitidos.includes(estatus) ||
61     excedeLimite
62   ) {
```

En esta función se agregan límites de caracteres a las casillas del formulario, esto debido a que algunas de las variables de usuario son de tipo `varchar`, que permite al oficial ingresar una cantidad variable de texto, pero en casillas como `rfc` solo ocupamos 18 caracteres, por lo que se limita la extensión de las entradas posibles

Convenciones de código del equipo:

Durante el avance de diseño de solución de nuestro equipo donde teníamos que declarar que convenciones utilizaremos durante el desarrollo de nuestro proyecto, las convenciones declaradas en el avance son las siguientes:

1. Variables y Funciones: en variables y funciones se hace el uso de la convención de escritura camelCase, que se caracteriza por ser una conexión de dos palabras, donde la primera palabra empieza en minúscula y la segunda palabra empieza con mayúscula, esto resulta en una mayor claridad al momento de leer el código. Además de camelCase declaramos que no se van a utilizar nombres ambiguos o demasiado cortos, para mejorar la comprensión y claridad al leer el código.
2. Strings: En los casos en donde se desplieguen textos se escribirán utilizando doble comilla (“”)
3. Segmentos de HTML: El código hecho en HTML debe estar bien estructurado, donde los diferentes segmentos deben estar separados y con un comentario en la línea superior a la línea de declaración de un segmento para mantener claridad.

Reflexión: Durante esta unidad de formación, aprender a desarrollar una aplicación web ha sido un reto importante, ya que inicié con poco conocimiento previo sobre la creación de módulos, funcionalidades y componentes de un sistema. Sin embargo, conforme avanzaron las semanas, pude comprender mejor la estructura de una aplicación, la relación entre el front-end, el back-end y la base de datos, así como la importancia de mantener un código ordenado.

Tanto mi equipo como yo tomamos como base los ejemplos desarrollados en clase para adaptarlos e implementarlos dentro de nuestro proyecto. Esto nos permitió avanzar de forma progresiva, resolver problemas reales del sistema y comprender cómo los conceptos vistos en clase se aplican en una solución funcional.